Report: Navigation and SLAM internship

Author: Francesca Camarena · **15 h check:** reached during Part 2, task 4 (precise arrival): tasks 1–3 coded and tested, the ground truth and baseline runs of task 4 started · **Finish:** ≈ 21h15: after the 15 h mark, task 4 tuning and task 5 (obstacle) coded and tested, the one-week plan written, the README and the report polished and summarized

I. SLAM on the X2

Explain how you would build a map with the X2 and keep it localized on that map, using the documentation, the robot's URDF and the latest SDK.

Building the map. The robot first has to learn what the office looks like. It walks through it once, slowly, while a laser scanner on its chest measures the distance to the walls and furniture around it (A). Because the scanner moves with the robot's body, the robot's model tells where the scanner is at every moment (B), and the manufacturer's software gives access to the scanner and moves the robot (C). A mapping method stitches the scans together into one map of the office, which is saved (D).

Staying localized. Afterwards, the robot keeps comparing what its scanner sees with the saved map, like recognising a room from its walls, to know where it is. Between comparisons it estimates how far it has walked, and the map corrects the small errors that build up. If people or moved furniture confuse it, it can find itself again from a known place, the reception (E).

A. Sensors

Which X2 sensors are useful for SLAM and navigation? Where are they on the robot, and what are their limits (field of view, blind spots, range)?

The sensors useful for SLAM and navigation, ranked by need; their locations are marked on the robot (see Fig. 1):

Rank	Sensor	Where (URDF)	Field of view	Range, rate	Limits
1	3D LiDAR RoboSense E1R	Chest, *torso_link*, 1.02 m high, level	120° × 90°	0.1–30 m, 10 Hz	Front only; floor hidden within 1.0 m; glass, mirrors; arms can occlude
2	IMUs (6-axis)	Inside the LiDAR; torso, pelvis	–	200 Hz (LiDAR), 500 Hz	Yaw drifts; gait vibration
3	Joint encoders	Every joint	–	Joint states	Leg odometry slips with the feet
4	RGB-D Orbbec Gemini 335	Head, 1.19 m, tilted 40° down	90° × 65°	0.26–3 m, 30 Hz	Short range; moves with the head
5	Front RGB (stereo, centre)	Head front, 1.29 m	156° × 120°	Images, 10 Hz	Images only, no depth from the SDK
6	Rear RGB	Back of the head	156° × 120°	Images, 10 Hz	Images only

Essential no SLAM without it · **Recommended** closes the blind strip in front of the feet · **Optional** images only

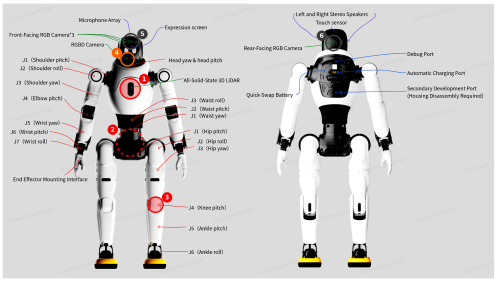


Fig. 1: Sensors on the X2 Ultra (numbers as in the sensor table).

Two blind spots remain (see Fig. 2): no range data in the 240° behind and beside the robot, and no sensor sees the floor within 0.44 m.

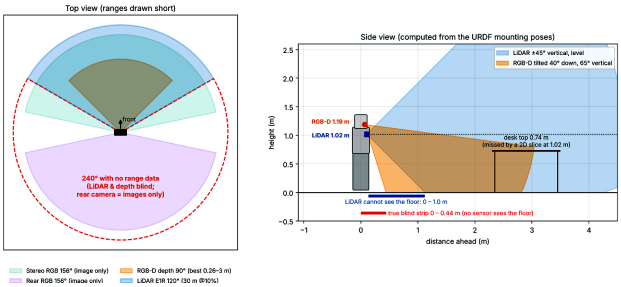


Fig. 2: Sensor coverage: top and side views.

B. Robot model

Using the URDF and the coordinate-system section, describe the coordinate frames you need between the map and the sensors. Draw the frame tree.

The **URDF** is the manufacturer's description of the robot's body: its parts, how they are joined, and where each sits on the one it is attached to. Each part has a reference point, a **frame**, and the **coordinate-system section** of the docs shows where the sensors' frames are and how they were calibrated. Frames are needed because each sensor measures from where it is mounted; linking them converts its data into the office's frame.

Only a **subset** of the URDF's 30+ frames is needed: those connecting the map to the SLAM sensors. They are the office (*map*) and the walk's starting point (*odom*), computed by localization and odometry; the hips (*pelvis*), the root; the waist links and chest (*torso_link*), because the waist turns and carries the scanner; and the scanner with its IMU (*lidar_chest_front_link*). The head camera adds the two neck links. Arms, hands, legs and feet are left out: no SLAM sensor is attached to them, and walking is AgiBot's controller's job.

Fig. 3 shows the tree: blue links come from localization and odometry, red ones from the joint angles, grey ones are fixed mounts.

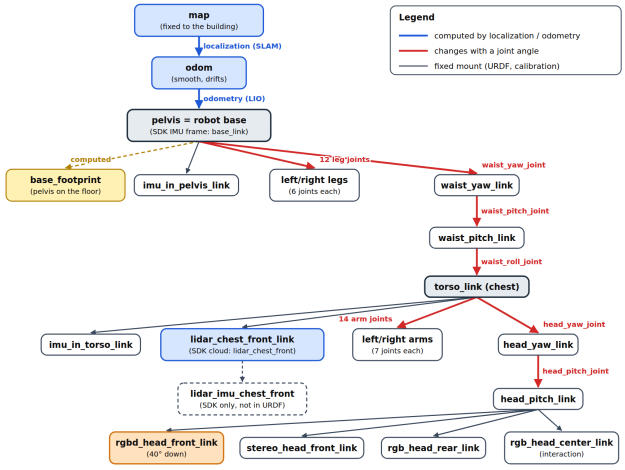


Fig. 3: X2 frame tree.

C. SDK interfaces

Which interfaces give you the LiDAR and IMU data, and which one makes the robot walk? Name them as they appear in the SDK.

An **interface** is the way a program talks to the robot without touching its drivers. On the X2 they are ROS 2 **topics**, continuous streams of data that a program reads or writes, and **services**, one-off requests that get an answer. The sensor data comes on topics:

Interface	Details
/aima/hal/sensor/lidar_chest_front/lidar_pointcloud	LiDAR point cloud, *sensor_msgs/PointCloud2*, 10 Hz
/aima/hal/sensor/lidar_chest_front/imu	IMU inside the LiDAR, *sensor_msgs/Imu*, 200 Hz
/aima/hal/imu/chest/state, */aima/hal/imu/torso/state*	Body IMUs, *sensor_msgs/Imu*, 500 Hz, QoS best effort

Walking uses one topic, the velocity command. Three services must be called once before it: they don't appear in the topic list because they are services, not topics (*ros2 service list* shows them), but without them the robot ignores the velocity commands. They put it in a standing mode, register the program as an allowed command source, and set the speed limits:

Interface	Details
/aima/mc/locomotion/velocity	Topic, message *McLocomotionVelocity*: forward, lateral and angular velocity, about 50 Hz
SetMcAction	Service: puts the robot in stable stand (*STAND_DEFAULT*)
SetMcInputSource	Service: registers your program as a command source (name *sdk_…*, priority 20–39)
SetMcLocomotionSpeedMode	Service: speed limits (LOW: −1.0 to +0.8 m/s forward)

D. Method

Which SLAM approach would you choose for the X2, and why? Compare it with one other approach. Say what the SDK's optional SLAM and navigation module offers and whether you would use it.

LiDAR-inertial odometry (FAST-LIO2), with a 2D grid for Nav2; AgiBot's modules only as a baseline (arrival radius ≥ 0.25 m).

SLAM approach	Benefits	Disadvantages
LiDAR-inertial odometry, FAST-LIO2 (chosen)	6-DoF; scans de-skewed with the IMU, so walking does no harm; 3D obstacles kept; light enough for the PC2	No loop closure (add a pose graph or use LIO-SAM); more integration work
2D graph SLAM, slam_toolbox	Native to Nav2, loop closure built in, well documented	Assumes planar motion: the horizontal slice tilts with the torso, walls smear; misses table tops
AgiBot SLAM + navigation modules (optional)	Mapping, stored maps, relocalization, point-to-point navigation with obstacle avoidance; tuned for the X2	Black box; arrival radius ≥ 0.25 m; must be enabled by after-sales support

E. Problems Name the 3 most serious problems you expect when doing SLAM and navigation on a walking humanoid in an office, ranked. For each one, say how you would detect it or reduce it.

The main problems, ranked: rank 1 is the most severe, because it corrupts the map that everything else uses.

Rank	Problem	How it is detected	How it is reduced
1	**Walking motion corrupts perception:** the torso sways and pitches at every step, so scans distort and a 2D slice tilts into the floor	Thick or doubled walls in the map; LIO and leg odometry disagree; IMU peak at the step frequency	IMU de-skewing (LIO); height-band projection in the map frame; map in the LOW speed mode, head still
2	**Imprecise velocity execution:** the walking interface is open-loop, and small commands (below about 0.1 m/s) are ignored	Log commanded against measured velocity; arrival error over repeated runs	Command at least 0.1 m/s and stop on the measured pose; align the heading 0.5 m before the goal, then walk straight in
3	**Localization loss in a changing office:** people, moved chairs, glass, and 240° with no range data	Low scan-match score; growing pose covariance; jumps in *map → odom*	Remove moving points (people detection); relocalize at a known pose (reception); keep-out zones on glass; update the map

F. Pipeline Draw your full pipeline, from the sensors to the walking commands.

Pipeline B takes the X2 from raw sensor readings to walking commands that bring it to a goal on the office map. It has five stages, numbered as in Fig. 4: ① sensors, ② state estimation, ③ map and localization, ④ navigation, and ⑤ the robot's walking control.

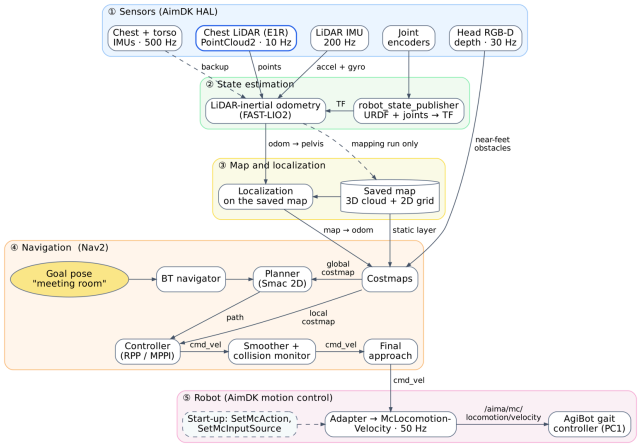


Fig. 4: Pipeline B, from the sensors to the walking commands.

II. SLAM and Navigation in Simulation

Build a map and navigate in it, in a ROS 2 simulation you set up yourself.

A. Setup Choose a simulated robot and an office-like indoor world. Explain your choice and give the exact versions you used (OS, ROS 2 distribution, simulator).

The table summarizes the technology stack: the version used and the reason for each choice.

	Choice	Version	Reason
Environment	Docker, Ubuntu container	Docker 29.4.0, Ubuntu 24.04.5	Jazzy has no packages for the laptop's Ubuntu 25.10; reproducible setup
Robot	TurtleBot3 Burger	turtlebot3_gazebo 2.3.7	Standard robot, 360° LiDAR (3.5 m), light to simulate
World	TurtleBot3 House	turtlebot3_gazebo 2.3.7	Most office-like stock world: rooms, doors, furniture
ROS 2	Jazzy	LTS to May 2029	Every package needed exists as a binary
Simulator	Gazebo Harmonic	gz-sim 8.15.0	Official pairing for Jazzy
SLAM	slam_toolbox (online async)	2.8.5	Nav2's default, loop closure, low CPU
Navigation	Nav2 (AMCL + RPP)	1.3.13	Standard stack; RPP instead of MPPI for a 4-core laptop

B. Mapping Build a map with a SLAM package by driving the robot around. Save the map and comment on its quality: what is good, what is wrong, and why.

The map was built with **slam_toolbox**, a graph-based SLAM: each new laser scan is matched to the previous ones, and when the robot returns to a place it has already seen, a loop closure corrects the drift that has built up. The robot followed a scripted route based on route D, the best of 6 options tested: it drives the middle of each room and pauses at viewpoints from which every wall is seen at least twice, drives the narrow rooms straight in and out, circles the south-east tables once, rounds its corners and turns gently, and returns to the start to close the loop. Fig. 5 compares the saved map with the true walls.

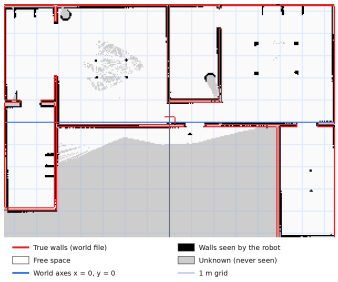


Fig. 5: Saved map against the true walls.

Grey inside the house	Walls found	Map wall cells within 10 cm	95th percentile	End error SLAM / odometry
2.0 % (1.7 m²)	96.3 %	95.8 %	10 cm	2.1 / 1.7 cm

What is wrong in the map, and why:

Problem	Why
South-west room drawn 10–20 cm off	2.2 m wide and 5 m long, it acts like a corridor: scans along it look alike, so the matcher can't fix the position along its length, and no loop pins its far end
Grey under the dining table (1.2 m²)	Table and chair legs block the beams; a cell turns free only after 2 beams cross it
Grey behind the north-room bin (0.2 m²)	No point of the route sees that corner
Not repeatable: clean in 1 of 4 runs	In the large north-east room most beams hit nothing within 3.5 m, so a wrong scan match can be accepted

C. Navigation Send the robot from a "reception" point to a "meeting room" point on your map. Explain the role of each component you used.

Nav2 answers three questions, *where am I*, *how do I get there* and *how do I move there safely*, each with its own components. The table lists them, with what each does and how it is set here.

Question	Nav2 component	What it does	Setting here
Where am I?	*map_server*	Loads the saved map and publishes it on */map*	*maps/ house.yaml*
	AMCL (Adaptive Monte Carlo Localization)	Matches each laser scan to the map and publishes *map → odom*, the robot on the map	Starts at reception; range 3.5 m
How do I get there?	Goal: *make go* or RViz	Sends the meeting-room goal and gets the result back	*config/ goals.yaml*
	BT navigator (Behaviour Tree)	Asks for a path every second, calls the recoveries on failure	Default tree
	Planner: NavFn (Navigation Function)	Lowest-cost path to the goal on the global costmap	Default
	Global costmap	Whole house: saved map, obstacles from the scan, safety margin	Robot radius 0.14 m
How do I move safely?	Controller: RPP (Regulated Pure Pursuit)	Follows the path; slows near walls and at the goal	Look-ahead 0.4 m, 10 Hz
	Local costmap	3 × 3 m around the robot, from the live scan	Robot radius 0.14 m
	Behaviour server	Recoveries when stuck: spin, back up, wait	Default
	Velocity smoother	Caps speed and acceleration, sends */cmd_vel* to the wheels	0.22 m/s, gentle

Fig. 6 shows how the information flows between these components.

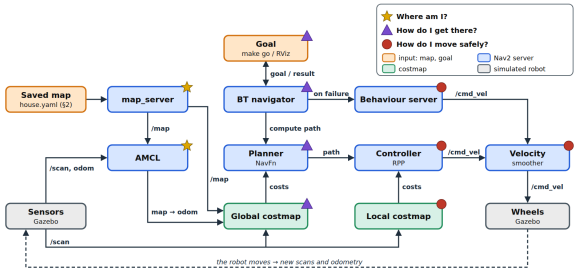


Fig. 6: How the information flows in Nav2.

A **costmap** is a grid laid over the map in which each cell gets a cost from how close it is to an obstacle. Nav2 builds it from the saved map plus the live laser scans, and adds a safety margin around every obstacle. The planner looks for the cheapest path through it, and the controller uses it to check for collisions (Fig. 7).

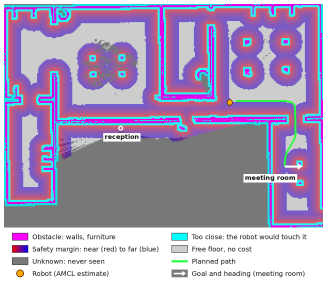


Fig. 7: Global costmap while driving to the meeting room.

D. Precise arrival Stop as close as possible to the goal, within 5 cm and 5°. Run it 5 times and report the errors in a table. Explain how you measured them and what you changed to get closer.

The error is measured against **Gazebo's true pose**: the robot's exact position and heading as the simulator itself knows them. The robot never sees it (it navigates on its own estimate, AMCL), so it is used only to measure. The 5 trips from reception to the meeting room were run twice: once with the settings of §C, and again after tuning. The error went from 10.8 cm / 37.5° to 14.3 cm / 1.2°: the heading target is met, but the position is limited by a 10 cm map offset in the meeting room.

Error	Measurement	Adjustment to reduce error
Arrival position	Distance between true pose and goal, *hypot(dx, dy)*	**Goal checker** (controller server): position tolerance 25 → 3 cm. **Path follower** (Regulated Pure Pursuit): starts slowing 1 m before the goal instead of 0.6 m, and crawls at 2 cm/s instead of 5. Still limited by a 10 cm map error
Arrival heading	True yaw − goal yaw, wrapped to ±180°	**Goal checker**: heading tolerance 14° → 3°. **Path follower**: the final turn on the spot slowed from 1.8 to 0.4 rad/s, because the robot overshot while braking
Localization	AMCL pose − true pose	**Localization** (AMCL): position updates every 5 cm and 3° instead of 25 cm and 11°: about 3 cm better, not kept because it caused a door failure
Stopping	AMCL pose − goal: where the robot *believes* it stopped	**Goal checker**: the tighter position and heading tolerances above

Columns: the robot's true final position minus the goal, along the map's x axis (east) and y axis (north), and in total; the true heading minus the goal heading; how far AMCL's estimate was from the true position; how far the robot believed it was from the goal; and the time of the trip from reception to the meeting room.

Before tuning:

Run	Position error along x (cm)	Position error along y (cm)	Position error, total (cm)	Heading error (°)	Localization error (cm)	Stopping error (cm)	Trip time (s)
1	-0.6	9.9	9.9	35.1	21.8	31.6	65.0
2	-0.4	14.3	14.3	37.1	17.3	31.6	67.1
3	2.9	7.6	8.1	33.8	13.3	21.1	68.9
4	-2.2	14.9	15.0	39.2	19.1	34.1	70.7
5	-1.0	6.8	6.9	42.2	19.2	26.1	71.3
Mean	-0.3	10.7	10.8	37.5	18.1	28.9	68.6
Max	2.9	14.9	15.0	42.2	21.8	34.1	71.3

After tuning (goal tolerance 3 cm / 3°, slower approach, final turn 1.8 → 0.4 rad/s):

Run	Position error along x (cm)	Position error along y (cm)	Position error, total (cm)	Heading error (°)	Localization error (cm)	Stopping error (cm)	Trip time (s)
1	1.6	-11.9	12.0	-0.4	16.1	4.1	99.1
2	2.1	-14.9	15.0	0.9	16.7	1.7	84.6
3	-1.4	-15.0	15.1	2.7	14.2	2.5	125.6
4	2.3	-17.6	17.8	2.0	16.7	3.3	92.6
5	1.6	-11.5	11.6	0.7	10.8	2.1	78.7
Mean	1.2	-14.2	14.3	1.2	14.9	2.7	96.1
Max	2.3	-17.6	17.8	2.7	16.7	4.1	125.6

E. Unexpected obstacle After mapping, place an object on the path. Show and explain what the robot does.

A box dropped 1.85 m ahead of the robot is marked in the costmaps and avoided by a new path 2.7 s later (Fig. 8). The saved map never changes.

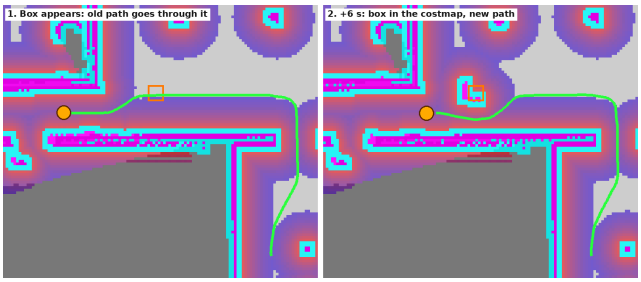


Fig. 8: Box before and after.

III. Plan for one more week

What you would do with one more week.

Goals: make mapping repeatable, guided by papers, and implement the Part 1 pipeline (Fig. 4) on the X2, one stage at a time:

Stage	Now (Part 2)	Change for the X2
Route	D.1, clean in 1 of 4 runs	Rules from the research, tested in Gazebo first
① Sensors	2D LiDAR, wheels, IMU in Gazebo	Chest 3D LiDAR + IMU, joints, RGB-D in MuJoCo, new sensor bridge
② Estimation	Wheel odometry	FAST-LIO2 (*odom* → *pelvis*), X2 URDF frames
③ Map, localization	slam_toolbox 2D map, AMCL	FAST-LIO2 3D map → 2D grid; scan-to-map localization
④ Navigation	Nav2, RPP, 2D costmaps	+ voxel layer (desk tops), final-approach node
⑤ Robot	*cmd_vel* → wheels	Adapter → *McLocomotionVelocity*, AgiBot walking controller

Task	What I do	Time	New tools
1. Research repeatable mapping	Papers on corridor slip, loop closure, route choice (active SLAM); write route rules and a 5-run test	6 h	–
2. Rules in the current setup	TurtleBot in Gazebo: one factor at a time, 5 runs each; remap without the 10 cm offset	6 h	*evo*
3. Stages ①, ⑤	X2 in MuJoCo: sensor bridge, walking adapter	9 h	MuJoCo, AgiBot X2 Sim
4. Stages ②, ③	FAST-LIO2 with the route rules, 2D grid, localization; 3 runs	9 h	FAST-LIO2
5. Stage ④	Nav2 voxel layer, final approach; 5 runs	6 h	Voxel layer
6. Results	TurtleBot vs X2; report	2 h	
Total		**38 h**	

Why MuJoCo: AgiBot ships the X2 and its walking controller for it; it needs a new LiDAR and ROS 2 bridge.

IV. Learning notes

What you read, what you tried, what failed, and roughly how much time each part took.

Part	Tried	Failed	Solution	Time
Part 1	Vocabulary, docs and URDF summary, similar projects	Too many new terms; projects too complex	Defined key terms only; Gemini Live conversations to clear doubts quickly; X2 docs and URDF as main source	~3h00
Setup	Docker, Gazebo, TurtleBot3, teleop, first node	Slow simulation; leftover servers	4 ms physics step, no Gazebo window; *make stop* before each start	~3h00
Mapping	slam_toolbox, scripted routes, 6 options	Incomplete first map; distorted routes; 1 of 4 clean	Scans every 0.2 m, deeper route, smooth turns; clean run kept	~5h15
Navigation	Nav2 servers only, RPP, scripted goals	Stuck in a door; preempted goal	Shorter look-ahead, slower near walls; one session at a time	~2h45
Precise arrival	Ground truth, baseline, 4 tuning steps	36° overshoot; map offset	Slower final turn (heading within 3°); map offset needs a better map	~3h25
Obstacle	Box on the path, blocked door	Box out of costmap range in the first try	Box dropped by the robot's position	~1h50
Report and README	Polished the README and the report; 3-page summary	Fixed-size pages left blank space; PDF export hung	Flowing A4 layout; export fixed	~2h00
Total				**≈ 21h15**

V. Resources

Question	Resources
General reading (learning notes)	Unitree G1 LiDAR, SLAM, navigation and control (sentdex) · What Is ROS2? Framework overview (Raymond Andrade) · Learn ROS2 practically with a real robot (Robotisim) · AimDK: get the SDK · AgiBot X2 wiki (Feishu)
Part 1, overview	AimDK X2 documentation (v1.1.0) · On-board computers · Quick start: prerequisites · Changelog
P1-Q1 Sensors	Sensor overview · Official X2 URDF / MuJoCo model · RoboSense E1R specifications · Orbbec Gemini 335 specifications
P1-Q2 Robot model	Coordinate system, URDF and calibration · Official X2 URDF / MuJoCo model
P1-Q3 SDK interfaces	Sensor interfaces · Locomotion control · Motion mode switching · MC control signal (input sources) · Python examples
P1-Q4 Method	SLAM module (optional) · Navigation module (optional)
P1-Q5 Problems	Locomotion control · Sensor overview
P1-Q6 Pipeline	Sensor interfaces · Locomotion control · Nav2 navigation concepts
P2-Q1 Setup	ROS 2 distributions · Gazebo and ROS 2 pairings · TurtleBot3 e-Manual, simulation · turtlebot3_simulations (jazzy) · rosdistro issue #53937
P2-Q2 Mapping	slam_toolbox · Nav2 map_server · ROBOTIS e-Manual, SLAM · turtlebot3_house model.sdf · MathWorks, lidar SLAM
P2-Q3 Navigation	Nav2 navigation concepts · Costmap 2D · Inflation layer · Lifecycle manager · AMCL · NavFn · Regulated Pure Pursuit · BT navigator · Velocity smoother · Simple Commander API · nav2_params.yaml 1.3.13
P2-Q4 Precise arrival	Regulated Pure Pursuit · Simple goal checker · Velocity smoother · AMCL · ros_gz_bridge
P2-Q5 Unexpected obstacle	Costmap 2D, obstacle layer · Inflation layer · BT navigator · Behaviour server · ros_gz_sim create
Part 3 plan (reading list)	On degeneracy of optimization-based state estimation (Zhang et al., 2016) · Real-time loop closure in 2D LIDAR SLAM (Hess et al., 2016) · SLAM Toolbox: SLAM for the dynamic world (Macenski, Jambrecic, 2021) · A survey on active SLAM (Placed et al., 2023) · FAST-LIO2 (Xu et al., 2022) · LIO-SAM (Shan et al., 2020)